Operating System.

→ It is a link between the user and the hard ware.
→ It act as bridge to enable communication between the user and hardware, and between the user and application Software.

Types.

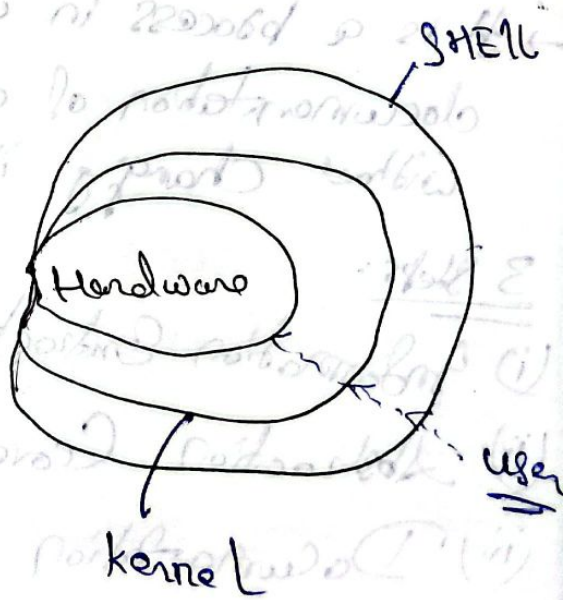
(1) SUI (Single user Interface)
→ An interface where only one user can work at a time.
→ eg: Android, MS Dos, iOS ... etc.

(2) MUI (multiple user Interface).
→ An interface where more than one user can work same time.
→ eg. Linux & unix etc.

* Parts of OS:

(a) Shell: (outer layer)
→ The user interface with this layer.

(b) kernel:
→ Internal layer of the OS
→ Interacts with the hardware.
→ core of OS.



* Interface of OS:

(i) CUI/CLI :-

→ character user interface / command line interface
→ To perform any task, the desired command must be typed.
ex. ms Dos, Unix etc.

(ii) GUI :-

→ Graphical user interface
→ Desired graphical tool is used.
→ For any OS to be considered GUI, it must follow 4 conditions. WIMP (window, icon, menus, Pointer)
→ ex. Android, ms windows, ios, macintosh

* Function of OS:-

(1) Process management :-

→ The main task of the OS process management module is to create and delete processes.
→ The OS schedules CPU time for all available process in memory. This is called process scheduling or scheduling

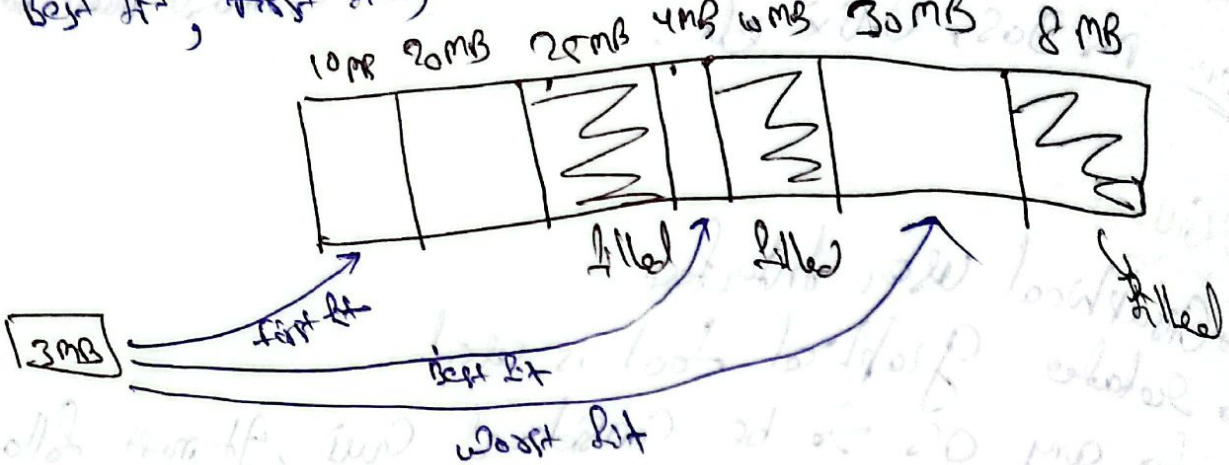
(2) File management :-

→ The OS modules creates, deletes and retrieve files.
→ Eg: Desktop.ttf
filename extention.
→ ex. FAT, NTFS, APFS, HFS, exFAT, nfs, zfs.

(3) Memory Management :-

→ OS, ~~Memory~~ Responsible for memory allocation & deallocation.

Ex: Best fit, first fit, worst fit.



(4) Security management :-

→ The OS is also responsible for the security of the Computer System and for preventing unauthorized access.

→ For security passwords are used for files, folder and system entry.

(5) Input-output management :-

→ Through its module, the OS manages input-output devices and operations.

(6) Integrated management :-

→ Data is managed by the OS in an integrated form. So that the working of one data does not disturb the extra processing time of another.

(7) Command management :-

→ The OS responds to the command / instruction given by user

(8) Disk management :-

→ The OS also manages the secondary memory / storage / hard disk in the Computer System.

(9) Error management :-

→ The OS also manages the error occurs in the Computer System.

Ex If the printer runs out of paper.

* Techniques of OS :-

1) Batch processing OS :-

→ Serial OS.
→ Instead of processing the same type of jobs seperately, they are processed together in a single batch.
→ There is no direct connectivity b/w the user and the system, So it is called an offline processing system.
→ used in mainframe or when large data is handled.
→ All tasks / jobs are processed sequentially.

(2) Multitasking OS :-

→ A technique in which the OS can handle multiple tasks at the same time.

→ While the OS is executing one task, it can allocate resources to another task. This is called time slicing.

→ Ex. Android, windows.

→ C.P.U time is divided among tasks.

(3) Multiprogramming OS :-

→ A technique in which multiple programs are loaded into memory at the same time, but only one program uses the CPU at a time.

→ minimize the C.P.U idle time.

→ Ex : Linux OS

→ Due to multiprogramming, multitasking is possible.

(4) Multiprocessing :-

→ When a system uses multiple CPUs to process tasks simultaneously, it is called multiprocessing.

→ Types

(a) Symmetric :- All CPU have equal access.

(b) Asymmetric :- One CPU is primary, others are secondary.

(5) RTOS (Real time)

→ manages real time applications.

→ Time. Critical and must be completed within a specific timeframe

Types...

Feature	Hard RTOS	Soft RTOS
1. Deadline.	Strict deadline	Flexible deadline
Impact of deadline miss	System fails	System doesn't fail, only performance is affected
Example	medical devices, aviation, Nuclear	Streaming, online gaming.
Response time	Deterministic	Non-deterministic
Use case	For System that are time-Critical	Not time-Critical.

(6) Time Sharing OS :-

→ A single processor / CPU is shared among multiple processes.
→ The CPU is allocated to all processes for a short time, called a time slice or Quantum time.
→ Each process can execute during its time slice, and as soon as the time slice ends, it goes to the ready state and wait for its next time slice.

(7) Distributed OS :-

→ multiple independent computers are connected / integrated → to work as a single system.
→ eg. facebook, Instagram, Google.

* Process Scheduling Algorithm :-

(1) fifo / fcfs :-

→ first come first serve.

- Turn Around time (TAT) = Completion time - Arrival time.
- Waiting time (WT) = TAT - Burst time.

↳ time taken by a process to complete or Execution time.

(1)

Process	Arrival time	Burst time
P_1	0	4
P_2	1	3
P_3	2	2
P_4	3	1

Sol^n.

Process	AT	BT	Starting	finish time	~~TAT~~	WT
P_1	0	4	0	4	4	0
P_2	1	3	4	7	6	3
P_3	2	2	7	9	7	5
P_4	3	1	9	10	7	6

Avg WT = $14/4 = 3.5$, Average TAT = $24/4 = 6$.

(2) SJF (Sortest Job first):-

→ The process with the shortest burst time is executed first.
→ It can be preemptive (SJN) or non-preemptive (SJF)
→ SJF minimizes the average waiting time.
→ In the SJF algorithm, Starvation can occur because if small processes keep coming, the larger processes may never get executed.

ex:

Process	Arrival time	Burst time
P_1	0	3
P_2	0	2
P_3	0	4

Process	AT	BT	Start time	Finish time	TAT	WT
P_2	0	2	0	2	2	0
P_1	0	3	2	5	5	2
P_3	0	4	5	9	9	5

* TAT = CT - AT
* WT = TAT - BT

AWT = $7/3 = 2.3$, ATAT = $\frac{16}{3} = 5.33$.

(3) Round Robin :-

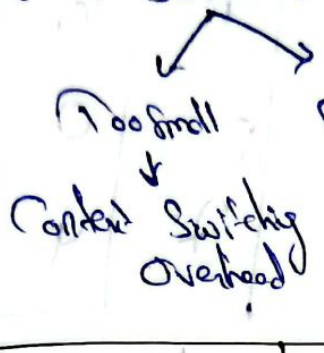
→ Each process is allocated a fixed time slot of the CPU, which is called a time slice or Quantum time.
→ If a process does not finish execution in its given time slice, it goes in the Queue for the next time.
→ This time slice is so small that all processes feel like only they are being processed.

* Advantages :-

→ Fair Scheduling.
→ No Starvation.
→ Best for time sharing systems.

Disadvantages :-

→ Time Quantum Selection tough.



Ex.

Process	Arrival time	Burst Time
P_1	0	5
P_2	0	4
P_3	0	2

Time Slice = 4ms.
Find out
(A) Average TAT.
(B) Average WT.
(C) Completion time of P_2?

Process	AT	BT	Completion time	TAT	WT
P_1	0	5	11	11	6
P_2	0	4	8	8	4
P_3	0	2	10	10	8
				29	18

A = 29/3
B = 18/3
C = 8

(4) Priority Scheduling :-

→ Each process is given a priority number as input.
→ Processes with higher priority are executed first
→ If processes have the same priority, the priority scheduling works like fcfs.
→ If the priority of a process changes, it is called preemption.
→ Efficient for real-time
→ Starvation can occur, as lower priority processes may never get executed.

Q.

Process	AT	BT	PI	ST	CT	TAT	WT
P_1	0	4	2	0	4	4	0
P_3	4	2	1	4	6	2	0
P_2	2	3	3	6	9	7	4
						13	4

$\therefore$ Avg TAT $= \frac{13}{3}$, Avg. WT $= 4/3$.

Response time = Time when the process get the CPU − Time when the process entered memory.

(5) SRTF (Shortest Remaining Time First)

→ Preemptive version of SJF.

→ The CPU is given to the process with the shortest remaining burst time.

→ If a new process arrived with a burst time less than the current process, the current process is preempted.

→ Reduce average waiting time.

→ Starvation possible.

Q.

Process	AT	BT
P_1	0	8
P_2	1	4
P_3	2	2

Completion time of P_1 = ?

Sol: 0 — P_1 — 1 — P_2 — 2 — P_3 — 4 — P_2 — 7 — P_1 — 14

P_1: 14 − 0 = 14; P_2: 7 − 1 = 6; P_3: 4 − 2 = 2

[Completion time $P_1 = 14$]

(6) MQS (multilevel Queue Scheduling)

→ multiple process are divided into different Queues. (based on priority / burst time), and each Queue uses a different Scheduling algorithm

→ Difficult to manage and implement.

Eg:
- Queue 1 = P_1 & P_2 (round robin) = High priority.
- Queue 2 = P_3 (fcfs) = low priority.
- Execution order :- $P_1 \rightarrow P_2 \rightarrow P_3$.

(7) MLFQ (multi level feedback Queue Scheduling)

→ Advanced version of MQs.
→ Process Queue can change based on behaviour.
→ Process that uses more CPU = low priority.
→ Process that gives I/O Quickly = high priority
→ used in modern OS (Linux, Windows)
→ No starvation, very complex, Hard to tune parameter.

(8) HRRN (Highest Response Ratio Next)

→ Non preemptive
→ used to avoid starvation.
→ Response ratio = $\frac{(\text{Burst time} + \text{waiting time})}{\text{Burst time}}$
→ High response ratio = executed first.
→ Avoid starvation
→ Calculation overhead.

(9) fss (Fair Share Scheduling)

→ The CPU divided among users, not among processes
→ Each user gets an equal CPU share, no matter how many processes they have
→ User-based Scheduling
→ User in multi-user systems

eg:- User 1 = 3 process.
User 2 = 10 processes
CPU allotment = 50:50.

* Differences :-

Algorithms	Preemption	Starvation	Complexity.
fcfs	x	✓	L
SJF	x	✓	M
SRTF	✓	✓	H
RR	✓	x	M
PS	✓/x	✓	M
MQS	✓/x	✓	H
MLFQ	✓	x	VH
HRRN	x	x	M
fss	✓	x	VH

* Deadlock:-

When 2 or more processes hold some resources (RAM, Scanner, Disk etc) and demand some other resources, which are held by other processes, then an infinite wait condition is created, which is called deadlock.

Conditions:-

(i) Mutual Exclusion:-

→ At least one resource must be held in non sharable mode. i.e only one process can use the resource at any given time.

(ii) Hold and wait:-

→ A process is holding at least one resource and it waiting for other resources that are held by other processes.

(iii) No preemption:-

→ When a process is holding a resource, it cannot be forcefully released. It can only be released voluntarily by the process after execution is complete.

(iv) Circular wait:-

→ A closed chain of processes exists, where each process is waiting for a resource held by the next process in the chain. For ex: P_1 is waiting for P_2, P_2 is waiting P_3, and P_3 is waiting for P_1.

* Deadlock Handling methods:-

(1) Deadlock Prevention:-

→ Make atleast one of the 4 condition fail.
→ If circular wait is stopped, then deadlock does not occur.

(2) Deadlock Avoidance:-

→ In this, before the OS allocates a resource to any process it check whether the system will go into an "unsafe state" or not.

→ Use Banker's Algorithm.
↳ check for safe and unsafe state.

Resource → Process → Avoid, Prevent.

(3) Deadlock Detection & Recovery:-

→ Deadlock is detected using a cycle detection algorithm.
→ Recovery can be done by killing/terminating the process or by preempting (forcibly taking) the resources.

(4) Deadlock Ignorance:-

→ Ostrich Algorithm.
→ Mostly used in windows & Linux.
→ It is assumed that deadlock will almost never occur, and if deadlock is detected, the system is simply restarted.

PCB (Process Control Block)

→ Every process has a PCB/ID card/ resource, which is maintained by the OS.

Parts:

(a) Process ID
(b) State
(c) Program Counter
(d) CPU register
(e) Scheduling info
(f) memory management info
(g) I/O status info.

Process States:

(1) New:
→ Process created for the first time.
→ Not yet in main memory.

(2) Ready:
→ When the process is loaded into RAM and is waiting to get the CPU
→ multiple processes can be in Queue, and the scheduler decides which process will be allocated the CPU next.

(3) Running:
→ When the CPU is executing the instruction of a process. It is in the Running state.

(D) Waiting / Block State :-

→ If a process needs I/O operation, it leaves the CPU and goes into the Waiting State.

→ Once the input-output work is done, the process returns to the ready State.

(E) Terminate / Exit :-

→ When a process is fully executed, the OS terminates it and removes it from memory.

Note :- When RAM becomes full, the OS shifts some processes to the hard disk.

(F) Suspend - ~~Ready~~ wait :-

→ If a process is in the waiting State and there is a shortage of RAM, it is shifted to the hard disk.

(G) Suspend - Ready :-

→ If the ready Queue (RAM) is full, the process is shifted to the hard disk.

* Context Switching :-

→ CPU switched from one process to another process.

→ During this, when the CPU drops PL, it saves its current details in its PCB.

* Scheduler:-

(i) long term (Job Scheduler):-

→ Decides which process will be brought from the Hard disk to RAM.

(ii) Short term (CPU Scheduler):-

→ fastest
→ Decides which process will go from the ready state to the running state.

(iii) Medium term:-

→ Responsible for swapping
→ If RAM is full, it sends process to the secondary memory.

* Interrupt:-

→ Signal / Event.
→ It tells the CPU to stop the current work and do some other urgent work first.
→ By default,
  → control goes to the OS
  → the OS handles the interrupt.
→ Interrupts are always handled in privileged mode / Kernel mode.

Types:-

(A) Hardware Interrupt:-

→ Generated by an external device.
→ Comes from outside the CPU.
→ Sources:- keyboard, printer etc.
→ Asynchronous, means it can occur at any time

(B) Software Interrupt:-

→ Generated by program itself.
→ used to access OS services.
→ Or read(), write(), open().
→ Synchronous means it occurs after instruction
→ Predictable. eg:- System call.